

MP1 Part 2: Build a Data Loaders Module

Continue working in the same `mp1-data-pipeline` repository from Part 1.

In Part 2, you will extend your command-line pipeline by adding a separate module for loading different data file formats. Your pipeline will now load the input file after validating it.

Project Structure

At the end of Part 2, your repository should contain:

```
mp1-data-pipeline/
├── .gitignore
├── pipeline.py          (updated)
├── data_loaders.py      (new)
├── requirements.txt     (new)
├── fixtures/           (new)
│   ├── sample.csv
│   ├── sample.json
│   └── sample.yaml
```

- `pipeline.py` — update your existing pipeline to use the data loaders.
- `data_loaders.py` — new module containing functions for loading different file formats.
- `fixtures/` — new directory containing sample files for testing.
 - These are small files used to test your data loaders, so do not add `fixtures/` to `.gitignore`. Make sure to git add and commit them.

Getting Started

1. Navigate to your existing repository:

```
cd mp1-data-pipeline
```

2. Set up and activate a virtual environment as practiced in class:

Note: We will cover virtual environments on Friday. You may skip this section for now and return to it after class.

```
python -m venv venv

# Mac/Linux
source venv/bin/activate

# Windows
venv\Scripts\activate
```

Install the required packages and create `requirements.txt`:

```
pip install pandas pyyaml  
pip freeze > requirements.txt
```

Make sure `venv/` is included in `.gitignore`. Do not add `requirements.txt` to `.gitignore`.

3. Make sure your Part 1 work is committed:

```
git status
```

4. Create a feature branch for Part 2:

```
git switch -c feature/data-loaders
```

5. Create the following new files:

```
data_loaders.py  
fixtures/sample.csv  
fixtures/sample.json  
fixtures/sample.yaml
```

Reuse the sample files from your Class 4 exercise. Copy them into `fixtures/` and rename them as follows:

Class 4 file	Part 2 fixture
<code>data/sample.csv</code>	<code>fixtures/sample.csv</code>
<code>data/sample.json</code>	<code>fixtures/sample.json</code>
<code>data/sample.yaml</code>	<code>fixtures/sample.yaml</code>

Starter Code

```
# data_loaders.py  
from pathlib import Path  
  
import logging  
import pandas as pd  
import json  
import yaml
```

```
# Do not call logging.basicConfig() here.
# Use the logging configuration from Part 1.
logger = logging.getLogger(__name__)

def load_csv(filepath):
    """Load a CSV file into a DataFrame.
    filepath is a Path object.
    """
    pass

def load_json(filepath):
    """Load a JSON file into a Python object (dict or list).
    filepath is a Path object.
    """
    pass

def load_yaml(filepath):
    """Load a YAML file into a Python object.
    filepath is a Path object.
    """
    pass

def load_data(filepath):
    """Load a file based on its extension.
    filepath is a string, such as 'fixtures/sample.csv'
    """
    pass
```

Required Functions

1. `load_csv(filepath)`

Load a CSV file into a pandas DataFrame.

Requirements:

- Use `pd.read_csv()`.
- Log an **INFO** message after successfully loading the file.
- Include the number of rows in the log message.
- Return the DataFrame.

Example:

```
INFO      Loaded CSV file: fixtures/sample.csv (5 rows)
```

2. `load_json(filepath)`

Load a JSON file — the same way you'd parse a typical API response.

Requirements:

- Use `json.load()`.
- Log an `INFO` message after successfully loading the file.
- Return the resulting Python object (e.g. dictionary).

Example:

```
INFO      Loaded JSON file: fixtures/sample.json
```

3. `load_yaml(filepath)`

Load a YAML file into a Python object.

Requirements:

- Use `yaml.safe_load()`.
- Log an `INFO` message after successfully loading the file.
- Return the resulting Python object (e.g. dictionary)

Example:

```
INFO      Loaded YAML file: fixtures/sample.yaml
```

4. `load_data(filepath)`

Automatically select the appropriate loader based on the file extension.

Use `pathlib` to create a `Path` object and get the lowercase file extension.

After selecting the appropriate loader, pass the `Path` object (`path`) to that loader rather than passing the original filepath again. `pd.read_csv()` and `open()` can both accept a `Path` object.

For example, the CSV case should call:

```
load_csv(path)
```

Your function should support the following formats:

Extension	Loader	Result
<code>.csv</code>	<code>load_csv()</code>	DataFrame
<code>.json</code>	<code>load_json()</code>	Python object

Extension	Loader	Result
.yaml	load_yaml()	Python object

For an unsupported file format:

- Log an **ERROR** message.
- Raise a **ValueError**.

Example:

```
ERROR    Unsupported file format: .txt
```

Update **pipeline.py**

Update your existing **pipeline.py** so that it uses the new **data_loaders** module. The pipeline should now actually load the input file.

Task

1. Import **load_data()** from **data_loaders**.
2. After successful validation,
 1. Call **load_data()** inside a **try** block and store the returned value in a variable called **data**.
 2. Catch **ValueError** in an **except** block and exit the program using **sys.exit(1)**.

Example Output

For a CSV file:

```
(venv) $ python pipeline.py --input fixtures/sample.csv --output clean.csv
--verbose

10:23:01 DEBUG    Arguments parsed: input=fixtures/sample.csv,
output=clean.csv format=csv
10:23:01 INFO     Input file validated: fixtures/sample.csv
10:23:01 INFO     Loaded CSV file: fixtures/sample.csv (5 rows)
```

When a file is missing:

```
(venv) $ python pipeline.py --input fake.csv --output output.csv

10:23:15 ERROR    Input file not found: fake.csv
```

When an existing file has an unsupported format:

```
(venv) $ python pipeline.py --input pipeline.py --output output.csv  
10:24:03 ERROR    Unsupported file format: .py
```

After completing Part 2

Your repository should contain:

```
mp1-data-pipeline/  
├── .gitignore  
├── pipeline.py  
├── data_loaders.py  
├── requirements.txt  
├── fixtures/  
│   ├── sample.csv  
│   ├── sample.json  
│   └── sample.yaml
```

Make sure that:

- `pipeline.py` continues to work after integrating the data loaders.
- Appropriate logging levels are used for successful loads and errors.
- All changes are committed to Git.
- Once completed, your feature branch is merged into `main`.
- The final `main` branch contains your completed Part 2 work.
- Your completed work is pushed to GitHub.